



PASHUPATASTRA

AI-Assisted Railway Maintenance Scheduling & Disruption Recovery

PHASE 1: DOMAIN REVIEW & RAILWAY DATA ARCHITECTURE

Presenter / Domain Lead: Darshini (feature/domain-data)

Integration & Backend Lead: Aryan (feature/backend-api)

Event: Smart India Hackathon 2026 — Phase 1 Parallel Kickoff

Status: Complete, Tested & Ready for Team Domain Freeze

1. The Railway Problem & Darshini's Mission

Context & Objectives

THE PROBLEM WE ARE SOLVING

- **Shared Track Capacity:** High-speed passenger express trains and heavy freight operate on the same tracks.
- **Maintenance Possessions:** Track renewals and repairs require complete line closure (possession block).
- **Scarce Windows:** Section controllers have very limited time slots (mostly nighttime and midday off-peak).
- **Daily Disruptions:** Unplanned rail fractures, OHE faults, and train delays frequently throw static schedules into chaos.

DARSHINI'S MISSION (DOMAIN + DATA)

- **Formal Domain Review:** Establish mathematical & operational railway rules in `DOMAIN_REVIEW.md`.
- **Protect Shared Contracts:** Maintain zero-breaking-change data contracts for the whole team.
- **Synthetic Data Generator:** Build seed-driven generator (`generator.py`) producing realistic corridors.
- **Provide Standard Fixtures:** Export Scenarios A, B, and C as benchmark datasets for all teammates.

2. System Pipeline: How Data Flows to the Team

Architecture & Handover

1	Domain & Data	Darshini (You)	Produces Corridor Topology, Asset Master, and Scenarios A / B / C
2	ML Priority & Risk	Ayush	Calculates explainable priority & risk scores from asset defect severity
3	CP-SAT Optimizer	Tyagi	Produces optimal schedule, enforces headway buffers, protects committed blocks
4	FastAPI Backend	Aryan	Exposes endpoints: POST /optimize, GET /health, POST /disrupt
5	Operations UI	Archit	Visualizes Corridor Timeline (Time vs. Tracks) & before/after recovery
6	Disruption Sim	Tirth	Injects disruptions (e.g. track unavailable) & triggers re-optimization loop

3. The 5 Core Domain Freeze Decisions

DOMAIN_REVIEW.md

Domain Area	Agreed Phase 1 Standard	Operational Reason & Team Impact
1. Track Modeling	Retain string track_id ("UP-1", "DOWN-1"). Attach geographic metadata in data layer.	Zero solver breaking changes. CP-SAT interval scheduling stays fast; UI gets rich section & direction grouping.
2. Maintenance Categories	6 Indian Railways work types: Track Renewal, Tamping, OHE, Signalling, Inspection, Emergency.	Matches physical operations, machinery needs, and precedence constraints.
3. Machinery Constraints	Shared heavy machines modeled via mutual_exclusion_group (e.g. BCM_01, CSM_01).	Guarantees a machine is never scheduled in two places simultaneously, avoiding solver timeouts.
4. Train Timetables	Modeled as Discrete Possession Windows (Night, Midday, Evening slots) over 24h horizon.	Solver enforces all scheduled blocks lie strictly within valid possession slots.
5. Disruption Scenarios	4 deterministic disruption triggers: Track Blockage, Emergency Work, Window Curtailment, Machine Breakdown.	Enables reproducible PLAN → DISRUPT → RECOVER demo flow on stage.

4. Maintenance Work Types & 24-Hour Possession Slots Operational Specifications

6 WORK CATEGORIES (WorkType)

Category	Duration	Machinery / Gang	Priority
TRACK_RENEWAL	180 – 240m	BCM_MACHINE_01	0.85 – 0.95
BALLAST_TAMPING	90 – 150m	CSM_TAMPER_01	0.70 – 0.85
OHE_MAINTENANCE	75 – 120m	TOWER_WAGON_01	0.65 – 0.80
SIGNALLING_INTERLOCK	60 – 120m	S_T_GANG_01	0.60 – 0.75
ROUTINE_INSPECTION	45 – 90m	Manual Trolley	0.35 – 0.50
EMERGENCY_REPAIR	90 – 150m	Flying Squad	0.95 – 0.99

24-HOUR POSSESSION WINDOWS (1440 MIN)

- 🌙 **Night Traffic Block (00:30 – 05:00 | 0030–0300 min):**
Heavy track renewals (CTR), ballast cleaning, and tamping.
- 🌞 **Midday Maintenance Slot (11:30 – 14:30 | 0690–0870 min):**
USFD flaw inspection, OHE power blocks, and tower wagon runs.
- 🌃 **Evening Off-Peak Slot (21:00 – 23:30 | 1260–1410 min):**
Signalling point machines and track circuit overhauls.
- 🕒 **Headway Buffer:** 15 minutes safety separation between consecutive blocks on the same track.

5. The 3 Synthetic Scenarios Ready in fixtures/

Benchmark Datasets

SCENARIO A: BASELINE

corridor_a_blocks.json

- **Corridor:** 2 Tracks (UP-1, DOWN-1)
- **Candidates:** 12 Blocks
- **Status:** 100% Feasible Schedule
- **Dependencies:** BLK-003 (Renewal) → BLK-004 (Tamping)
- **Machines:** BCM & CSM Tamper
- *Ready for Tyagi's CP-SAT solver.*

SCENARIO B: DENSE TRUNK

corridor_b_dense.json

- **Corridor:** 4 Tracks (UP-1/2, DOWN-1/2)
- **Candidates:** 24 Blocks
- **Saturation:** High-density traffic
- **Machines:** 4 competing machine groups
- **Purpose:** Tests solver trade-offs, priority ranking, and capacity limits.

SCENARIO C: DISRUPTED

corridor_c_disrupted.json

- **Corridor:** 2 Tracks | 14 Candidates
- **Committed Blocks:** BLK-001 & BLK-002 locked on DOWN-1 (01:00-03:15)
- **Disruption:** TRACK_UNAVAILABLE on UP-1 (02:00-05:00)
- *Ready for Tirth's recovery demo.*

6. Exact Handover for Each Teammate

Team Integration

Teammate	Workstream & Branch	What Darshini Has Prepared for You
Aryan	Backend & Integration (feature/backend-api)	DOMAIN_REVIEW.md complete; shared schemas in contracts/schemas.py locked for FastAPI integration.
Tyagi	CP-SAT Optimizer (feature/optimizer)	Use corridor_a_blocks.json to test baseline solve; machine exclusion groups & headway buffers active.
Ayush	ML Risk Scorer (feature/ml-scorer)	backend/app/data/models.py provides Asset with criticality, condition_score, and defect_severity.
Tirth	Simulation (feature/simulation)	corridor_c_disrupted.json has 2 pre-committed blocks ready for your TRACK_UNAVAILABLE test.
Archit	Frontend (feature/frontend)	Load corridor_a_blocks.json directly into React to render the Corridor Timeline UI immediately.

7. Automated Verification & Delivery Proof

Test Suite Results

10/10 UNIT TESTS PASSED (0.049s)

- ✓ `test_seed_reproducibility`
- ✓ `test_different_seeds_produce_variation`
- ✓ `test_track_assignment_validity`
- ✓ `test_time_window_consistency`
- ✓ `test_dependency_integrity`
- ✓ `test_work_types_are_valid_enums`
- ✓ `test_score_boundaries [0.0, 1.0]`
- ✓ `test_scenario_b_dense_properties`
- ✓ `test_scenario_c_committed_blocks`
- ✓ `test_json_fixtures_validity`

HOW TO RUN GENERATOR & TESTS

Run Interactive CLI:

```
python generate_data.py --scenario A
python generate_data.py --scenario B
python generate_data.py --scenario C
```

Run Automated Test Suite:

```
python -m unittest discover -s backend/tests -v
```